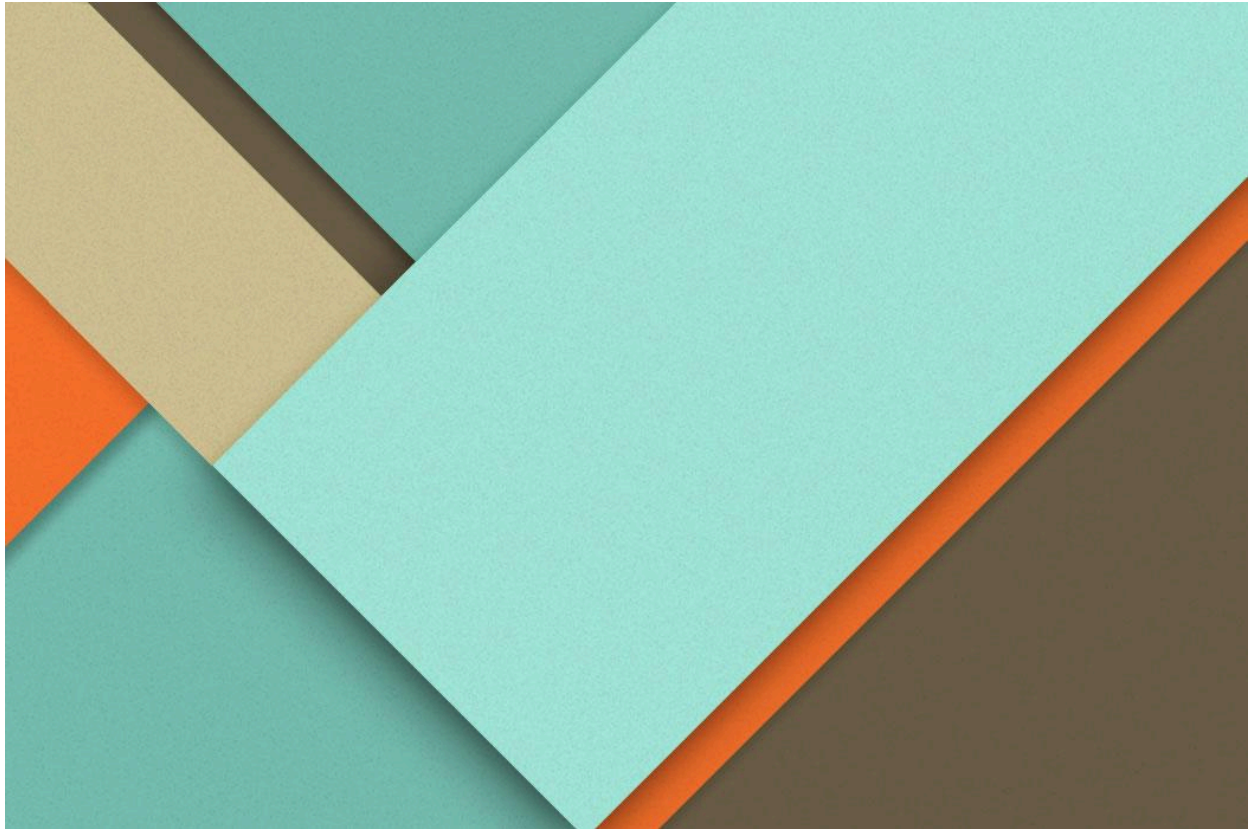




UD5: Práctica Aeropuerto

21/04/2026

José Manuel Sánchez Rosal

2º DAM

IES Antonio Gala

1400 Palma del Río (Córdoba)

Índice

Índice.....	1
1. Introducción.....	1
2. Instrucciones.....	1
3. Conclusiones.....	9

1. Introducción

El presente documento detalla el diseño, desarrollo e implementación del módulo personalizado "Aeropuerto" para el sistema ERP Odoo. El objetivo principal de este proyecto práctico es aplicar los conocimientos adquiridos sobre la arquitectura y el lenguaje de programación propios de los sistemas ERP-CRM. A través de este módulo, se facilita la gestión y el seguimiento de las operaciones de un aeropuerto, permitiendo el registro de la flota de aviones y el control detallado de las salidas. Para lograrlo, se han creado y configurado nuevos modelos de datos, vistas interactivas (formularios y listas), campos con cálculos dinámicos, restricciones de seguridad mediante roles de usuario y la generación automatizada de informes formales en formato PDF.

2. Instrucciones

Crea un módulo que gestione las salidas de los aviones en un aeropuerto con las siguientes características:

Tendremos dos objetos en nuestro sistema: salidas y aviones.

1) Los campos que tendrá el objeto aviones serán los siguientes:

- a. Nombre (Cadena de texto, obligatorio)
- b. Capacidad (entero, obligatorio)

c. Aerolínea (cadena de texto, obligatorio).

- Captura del código:

```
class Aeropuerto_avion(models.Model):
    _name = 'aeropuerto.avion'

    name = fields.Char(string="Nombre", required=True, help="Introduce el nombre del avion")

    capacidad = fields.Integer(string="Capacidad", required=True, help="Introduce la capacidad de pasajeros del avion")

    aerolinea = fields.Char(string="Aerolinea", required=True, help="Nombre de la Aerolinea")
```

- Captura de la ventana gráfica donde se ve el modelo creado:

Nuevo

Modelos

aeropuerto.avion

Descripción del modelo

aeropuerto.avion

Tipo

Objeto base

Modelo

aeropuerto.avion

En las aplicaciones ?

aeropuerto

Pedido ?

id

Modelo transitorio

☐

Campos

Permisos de acceso

Reglas de registro

Notas

Vistas

Nombre de campo	Etiqueta de campo	Tipo de campo	Requerido	Sólo lectura	Indexado	Tipo	
aerolinea	Aerolinea	carácter	☒	☐	☐	Campo base	
capacidad	Capacidad	entero	☒	☐	☐	Campo base	
create_date	Created on	fecha y hora	☐	☒	☐	Campo base	
create_uid	Created by	many2one	☐	☒	☐	Campo base	
display_name	Display Name	carácter	☐	☒	☐	Campo base	
id	ID	entero	☐	☒	☐	Campo base	
name	Nombre	carácter	☒	☐	☐	Campo base	
write_date	Last Updated on	fecha y hora	☐	☒	☐	Campo base	
write_uid	Last Updated by	many2one	☐	☒	☐	Campo base	

Añadir una línea

2) Para el objeto salidas tendremos los siguientes campos:

a. Avión (cadena de texto, obligatorio) será uno de los aviones ya registrados. Se tendrá que desplegar una lista con los aviones registrados.

b. Fecha (tipo fecha, opcional)

c. Hora de salida (tipo double)

Nota: una salida se corresponde con un único avión y un avión puede participar en varias salidas.

- Captura del código.

```
class Aeropuerto_salidas(models.Model):
    _name = 'aeropuerto.salida'

    avion = fields.Many2one("aeropuerto.avion", string="Selecciona un avion", required=True)
    fecha = fields.Date(string="Fecha de salida", help="Introduzca la fecha de salida")
    hora = fields.Float(string="Hora de salida", help="Introduzca la hora de salida")
```

- Captura de la ventana gráfica donde se ve el modelo creado.

Nuevo Modelos aeropuerto.salida

Descripción del modelo	aeropuerto.salida	Tipo	Objeto base
Modelo	aeropuerto.salida	En las aplicaciones ?	aeropuerto
Pedido ?	id		
Modelo transitorio	☐		

Campos Permisos de acceso Reglas de registro Notas Vistas

Nombre de campo	Etiqueta de campo	Tipo de campo	Requerido	Sólo lectura	Indexado	Tipo
avion	Selecciona un avion	many2one	☒	☐	☐	Campo base
calculado	Saludo	carácter	☐	☒	☐	Campo base
create_date	Created on	fecha y hora	☐	☒	☐	Campo base
create_uid	Created by	many2one	☐	☒	☐	Campo base
display_name	Display Name	carácter	☐	☒	☐	Campo base
fecha	Fecha de salida	fecha	☐	☐	☐	Campo base
hora	Hora de salida	número flotante	☐	☐	☐	Campo base
id	ID	entero	☐	☒	☐	Campo base
write_date	Last Updated on	fecha y hora	☐	☒	☐	Campo base
write_uid	Last Updated by	many2one	☐	☒	☐	Campo base

3) Crea un campo calculado en la vista lista del objeto salidas, en la que aparecerá lo siguiente: Si la hora está entre 8 y 12: "buenos días"; entre 12 y 18: "buenas tardes"; entre 18 y 9: "buenas noches"

- Captura del código.

```
calculado = fields.Char(string="Saludo", compute="_compute_saludo", store=True)

@api.depends('hora')
def _compute_saludo(self):
    for r in self:
        if 8 <= r.hora < 12:
            r.calculado = "Buenos días"
        elif 12 <= r.hora < 18:
            r.calculado = "Buenas tardes"
        else:
            r.calculado = "Buenas noches"
```

- Captura de la ventana gráfica donde se ve el modelo creado ya con el campo calculado

Nuevo Modelos aeropuerto.salida

Descripción del modelo	aeropuerto.salida	Tipo	Objeto base
Modelo	aeropuerto.salida	En las aplicaciones ?	aeropuerto
Pedido ?	id		
Modelo transitorio	☐		

Campos Permisos de acceso Reglas de registro Notas Vistas

Nombre de campo	Etiqueta de campo	Tipo de campo	Requerido	Sólo lectura	Indexado	Tipo
avion	Selecciona un avion	many2one	☒	☐	☐	Campo base
calculado	Saludo	carácter	☐	☒	☐	Campo base
create_date	Created on	fecha y hora	☐	☒	☐	Campo base
create_uid	Created by	many2one	☐	☒	☐	Campo base
display_name	Display Name	carácter	☐	☒	☐	Campo base
fecha	Fecha de salida	fecha	☐	☐	☐	Campo base
hora	Hora de salida	número flotante	☐	☐	☐	Campo base
id	ID	entero	☐	☒	☐	Campo base
write_date	Last Updated on	fecha y hora	☐	☒	☐	Campo base
write_uid	Last Updated by	many2one	☐	☒	☐	Campo base

4) Crea los menús siguientes colgando de un menú principal AEROPUERTO:

- aviones: Llevará la vista formulario (también se podrá acceder a la vista lista)
 - salidas: Llevará la vista lista (desde la que se podrá acceder a la vista formulario)
- Captura del código de las vistas, acciones de ventana y menús:

```
<odoo>
<data>
  <record id="view_aeropuerto_avion_list" model="ir.ui.view">
    <field name="name">aeropuerto.avion.list</field>
    <field name="model">aeropuerto.avion</field>
    <field name="arch" type="xml">
      <list>
        <field name="name"/>
        <field name="capacidad"/>
        <field name="aerolinea"/>
      </list>
    </field>
  </record>

  <record id="view_aeropuerto_avion_form" model="ir.ui.view">
    <field name="name">aeropuerto.avion.form</field>
    <field name="model">aeropuerto.avion</field>
    <field name="arch" type="xml">
      <form>
        <sheet>
          <group>
            <field name="name"/>
            <field name="capacidad"/>
            <field name="aerolinea"/>
          </group>
        </sheet>
      </form>
    </field>
  </record>
```

```

<record id="view_aeropuerto_salida_list" model="ir.ui.view">
  <field name="name">aeropuerto.salida.list</field>
  <field name="model">aeropuerto.salida</field>
  <field name="arch" type="xml">
    <list>
      <field name="avion"/>
      <field name="fecha"/>
      <field name="hora" widget="float_time"/>
      <field name="calculado"/>
    </list>
  </field>
</record>

<record id="view_aeropuerto_salida_form" model="ir.ui.view">
  <field name="name">aeropuerto.salida.form</field>
  <field name="model">aeropuerto.salida</field>
  <field name="arch" type="xml">
    <form>
      <sheet>
        <group>
          <field name="avion"/>
          <field name="fecha"/>
          <field name="hora" widget="float_time"/>
          <field name="calculado"/>
        </group>
      </sheet>
    </form>
  </field>
</record>

```

```

<record id="action_aeropuerto_avion" model="ir.actions.act_window">
  <field name="name">Aviones</field>
  <field name="res_model">aeropuerto.avion</field>
  <field name="view_mode">form,list</field> </record>

<record id="action_aeropuerto_salida" model="ir.actions.act_window">
  <field name="name">Salidas</field>
  <field name="res_model">aeropuerto.salida</field>
  <field name="view_mode">list,form</field> </record>

```

```

<menuitem id="menu_aeropuerto_root" name="AEROPUERTO" sequence="10"/>
<menuitem id="menu_aeropuerto_avion" name="Aviones" parent="menu_aeropuerto_root" action="action_aeropuerto_avion" sequence="10"/>
<menuitem id="menu_aeropuerto_salida" name="Salidas" parent="menu_aeropuerto_root" action="action_aeropuerto_salida" sequence="20"/>

```

- Captura donde se ven cómo quedan las vistas tipo list y formulario de cada uno de los modelos:

AEROPUERTO Aviones Salidas

Nuevo Nuevo ? ? x

Nombre ?

Capacidad ? 0

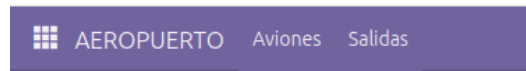
Aerolínea ?

AEROPUERTO Aviones Salidas My Company

Nuevo Salidas ? 1-2 / 2

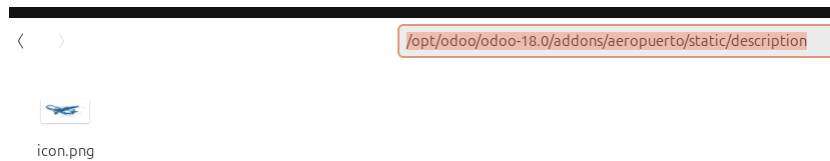
Selección un avion	Fecha de ...	Hora de salida Saludo
☐ Boeing 747	23/04/2026	10:00 Buenos días
☐ Boeing 434	26/04/2026	17:00 Buenas tardes

- Captura donde se ven los menús:

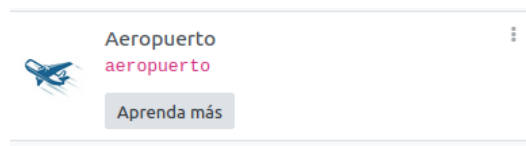


5) Utiliza un icono para ponerlo en el módulo.

- Captura donde se ve dónde metes el icono:



- Captura de la ventana gráfica donde se ve el icono:



6) Debes crear dos grupos, usuario e invitado. El primero dispondrá de control total sobre las dos tablas de las que consta nuestro módulo. El segundo grupo, tendrá los permisos de lectura únicamente sobre ambas tablas.

- Captura del código:

```
<odoo>
<data>
  <record id="module_category_aeropuerto" model="ir.module.category">
    <field name="name">Aeropuerto</field>
  </record>

  <record id="group_aeropuerto_invitado" model="res.groups">
    <field name="name">Invitado</field>
    <field name="category_id" ref="module_category_aeropuerto"/>
  </record>

  <record id="group_aeropuerto_usuario" model="res.groups">
    <field name="name">Usuario</field>
    <field name="category_id" ref="module_category_aeropuerto"/>
    <field name="implied_ids" eval="[(4, ref('group_aeropuerto_invitado'))]"/>
  </record>
</data>
</odoo>
```

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink

access_avion_invitado,aeropuerto.avion.invitado,model_aeropuerto_avion,group_aeropuerto_invitado,1,0,0,0
access_salida_invitado,aeropuerto.salida.invitado,model_aeropuerto_salida,group_aeropuerto_invitado,1,0,0,0

access_avion_usuario,aeropuerto.avion.usuario,model_aeropuerto_avion,group_aeropuerto_usuario,1,1,1,1
access_salida_usuario,aeropuerto.salida.usuario,model_aeropuerto_salida,group_aeropuerto_usuario,1,1,1,1
```

- Captura de la ventana gráfica donde se ven los grupos creados:

The screenshot shows the Odoo user configuration page for the 'Administrador' user. The interface is in Spanish and includes a top navigation bar with links like 'Ajustes', 'Opciones generales', 'Usuarios y compañías', 'Traducciones', and 'Técnico'. Below the navigation bar, there are tabs for 'Nuevos', 'Usuarios', and 'Administrador'. The main content area is divided into several sections: 'CONTACTO RELACIONADO', 'PERMISOS DE ACCESO', 'PREFERENCIAS', and 'SEGURIDAD DE LA CUENTA'. The 'PERMISOS DE ACCESO' section is active, showing the 'USUARIO' type and 'Tipos de usuario' (Usuario interno, Portal, Público). The 'ADMINISTRACIÓN' section shows 'Administración' and 'Ajustes'. The 'EXTRA RIGHTS' section shows 'Creación de contactos' and 'Multimoneda'. The 'OTHER' section shows 'Evitar la depuración de campos HTML'. The 'OTHER' section also includes a dropdown for 'Aeropuerto' (set to 'Usuario') and a checkbox for 'Múltiples compañías'.

7) Crea un informe pdf en el que se muestre cada una de las salidas y dentro de éstas todos los datos de los aviones.

- Captura del código del informe:

```
<odoo>
<data>
  <record id="action_report_salidas" model="ir.actions.report">
    <field name="name">Informe de Salidas</field>
    <field name="model">aeropuerto.salida</field>
    <field name="report_type">web-pdf</field>
    <field name="report_name">aeropuerto.report_salidas_template</field>
    <field name="report_file">aeropuerto.report_salidas_template</field>
    <field name="binding_model_id" ref="model_aeropuerto_salida"/>
    <field name="binding_type">report</field>
  </record>

  <template id="report_salidas_template">
    <t t-call="web.html_container">
      <t t-call="web.external_layout">
        <div class="page">
          <h2>Listado de Salidas y Aviones</h2>
          <table class="table table-sm">
            <thead>
              <tr>
                <th>Fecha</th>
                <th>Hora</th>
                <th>Avión</th>
                <th>Capacidad</th>
                <th>Aerolínea</th>
              </tr>
            </thead>
            <tbody>
              <t t-foreach="docs" t-as="salida">
                <tr>
                  <td><span t-field="salida.fecha"/></td>
                  <td><span t-field="salida.hora" t-options="{ 'widget': 'float_time' }"/></td>
                  <td><span t-field="salida.avion.name"/></td>
                  <td><span t-field="salida.avion.capacidad"/></td>
                  <td><span t-field="salida.avion.aerolinea"/></td>
                </tr>
              </tbody>
            </table>
          </div>
        </t>
      </template>
    </data>
  </odoo>
```



```
# always loaded
'data': [
    'security/security.xml',
    'security/ir.model.access.csv',
    'views/views.xml',
    'views/templates.xml',
    'reports/report_salidas.xml',
],
```

- Captura de la ventana gráfica donde se ve el informe en el que deben aparecer al menos dos salidas con todos los datos de sus correspondientes aviones:

Listado de Salidas y Aviones

Fecha	Hora	Avión	Capacidad	Aerolínea
22/04/2026	09:00	Boeing 747	455	Iberia
23/04/2026	14:00	Boeing 343	233	Ryanair
28/04/2026	00:00	Boeing 747	455	Iberia

3. Conclusiones

La realización de esta práctica ha permitido comprender en profundidad la arquitectura interna de un sistema ERP-CRM de clase mundial como Odoo. Se ha verificado de forma práctica cómo los diferentes componentes de software (modelos de Python, vistas en XML, archivos de seguridad CSV y plantillas QWeb) interactúan entre sí bajo el patrón Modelo-Vista-Controlador.

Se ha logrado añadir nuevas funcionalidades al sistema cumpliendo todos los requisitos planteados: extracción de información mediante campos relacionales, manipulación de datos con funciones de cálculo dinámico, y el blindaje de la información mediante políticas de acceso. El resultado final es un componente integrado, verificado y completamente funcional que extiende la capacidad original del ERP, demostrando que es posible adaptar el software a las necesidades específicas de un modelo de negocio real.